

Code Walkthrough

Exact-State Curie pNMR Shielding from Magnetic and Hyperfine Matrix Elements

1 What the script does

The script evaluates the **temperature-dependent Curie contribution** to the pNMR shielding tensor from state-resolved magnetic and hyperfine matrix elements. The input contains zero-field quasi-relativistic energies, matrix elements of

$$D_u = \frac{1}{2}(g_e S_u + L_u),$$

and hyperfine matrix elements separated into FC, SD and LI terms.

The main result is the 3×3 Curie shielding tensor

$$\sigma^{\text{Curie}}$$

reported in ppm. The corresponding Curie chemical-shift contribution is

$$\delta_{\text{Curie}} = -\sigma^{\text{Curie}}.$$

The code therefore computes a shielding contribution, not a complete experimental pNMR chemical shift. A full pNMR treatment may also contain temperature-independent/orbital terms and experimental referencing.

All generated files are stored in a subfolder next to the input data:

```
calculation/  
|-- molecule.data  
|-- pnmr_curie_supervisor_report_folder.py  
'-- report/  
    |-- molecule.curie_report.txt  
    |-- molecule.curie_summary.txt  
    '-- molecule.curie_pcs_field.cube
```

The long report is diagnostic, the summary is intended for quick reading, and the cube contains a Curie-only long-distance point-dipole field.

2 Theory implemented in the code

2.1 Exact-state Curie contribution

States are grouped into zero-field degenerate levels λ . Inside one level, a labels an individual state. At temperature T ,

$$\beta = \frac{1}{k_B T}, \quad Q = \sum_{\lambda} d_{\lambda} e^{-\beta E_{\lambda}},$$

and the probability of one state in level λ is

$$w_{\lambda a} = \frac{e^{-\beta E_{\lambda}}}{Q}.$$

The code evaluates

$$\sigma_{uv}^{\text{Curie}} = \sum_{\lambda a} w_{\lambda a} \left[-\beta \sum_{a' \in \lambda} \text{Re}(\langle \lambda a | H_{B,u} | \lambda a' \rangle \langle \lambda a' | H_{\mu,v} | \lambda a \rangle) \right].$$

The inner sum includes only states a' belonging to the same numerically degenerate level. This is why the degeneracy tolerance is a physically important setting rather than only a programming detail.

2.2 Magnetic-field and nuclear-moment operators

The input provides

$$D_u = \frac{1}{2}(g_e S_u + L_u),$$

and the code forms

$$H_{B,u} = D_u/c.$$

For each nucleus, the input header specifies

$$P = g_e g_N \beta_e \beta_N.$$

The nuclear-moment perturbation is constructed as

$$H_{\mu,v} = g_e \mu_B \frac{A_v}{P}.$$

The hyperfine matrix elements are not converted to MHz; A/P is used in the same internal convention in which the input was generated.

2.3 FC, SD and LI decomposition

The input contains

$$A = A_{\text{FC}} + A_{\text{SD}} + A_{\text{LI}}.$$

Because the Curie expression is linear in H_μ ,

$$\sigma_{\text{TOTAL}} = \sigma_{\text{FC}} + \sigma_{\text{SD}} + \sigma_{\text{LI}}.$$

LI here is the LI part of the hyperfine operator. It must not be interpreted as the complete temperature-independent orbital NMR shielding.

3 Settings: what can be changed

The user normally edits only the block near the beginning of the script:

```
1 HERE = Path(__file__).resolve().parent
2 DEFAULT_DATA_FILE = HERE / "h2o.casscf.rel.pnmr.data"
3 TEMPERATURE_K = 298.15
4
5 DEGENERACY_TOL_HA = 1e-8
6 THERMAL_REPORT_CUTOFF = 1e-8
7 DISPLAY_TOL_PPM = 1e-6
8 DOUBLET_POP_CUTOFF = 0.999999
9
10 GEOMETRY_FILE = None
11 CENTER_ATOM = 1
12 WRITE_CUBE = True
13 REPORT_DIR = "report"
14 CUBE_FILE_SUFFIX = ".curie_pcs_field.cube"
15 CUBE_SPACING_A = 0.4
16 CUBE_PADDING_A = 4.0
17 CUBE_CUTOFF_A = 1.0
18 CUBE_SIGN = "shielding"
```

Setting	Meaning
DEFAULT_DATA_FILE	Default input if no command-line file is supplied.
TEMPERATURE_K	Temperature entering $\beta = 1/(k_B T)$.
DEGENERACY_TOL_HA	States closer than this are grouped into one level. 10^{-8} Ha $\approx 2.2 \times 10^{-3}$ cm $^{-1}$.
THERMAL_REPORT_CUTOFF	Suppresses negligibly populated levels in path output.
DISPLAY_TOL_PPM	Suppresses numerically irrelevant printed components.
DOUBLET_POP_CUTOFF	Minimum ground-doublet population for doublet/ $1/T$ checks.
GEOMETRY_FILE	Optional external XYZ file. If None , geometry from .data is preferred.
CENTER_ATOM	1-based centre used only for the spatial long-distance cube.
REPORT_DIR	Name of the output subfolder created beside the input.
WRITE_CUBE	Enables/disables cube output.
CUBE_SPACING_A	Grid spacing in Å. Smaller values give a larger, smoother cube.
CUBE_PADDING_A	Extra grid margin around the molecule.
CUBE_CUTOFF_A	Radius around the centre where the $1/R^3$ singularity is replaced by zero.

Setting	Meaning
CUBE_SIGN	"shielding" writes σ ; "shift" writes $\delta = -\sigma$.

Physical constants such as k_B , c , g_e , μ_B and bohr-Å conversion should normally not be changed.

4 Input parsing

4.1 matrix_block

The low-level reader converts one matrix section into dictionaries indexed by Cartesian component and state pair:

```

1 def matrix_block(lines, i, n):
2     M = {u: {} for u in "xyz"}
3     count = 0
4     while count < n*n:
5         if i >= len(lines):
6             raise ValueError(...)
7         line = lines[i]
8         i += 1
9         if not line.strip():
10            continue
11        p, z = line.split(None, 2), PAIR.findall(line)
12        if len(p) < 2 or len(z) != 3:
13            raise ValueError(...)
14        a, b = int(p[0]), int(p[1])
15        for u, (x, y) in zip("xyz", z):
16            M[u][(a,b)] = complex(float(x), float(y))
17        count += 1
18    return M, i

```

A single value is accessed as

$$M[u][(a,b)].$$

The dictionary form is convenient because the physics functions naturally work with state labels rather than only contiguous matrix indices.

4.2 read_data

The main parser recognizes:

1. geometry,
2. number of states and state energies,
3. magnetic moment matrix elements,
4. one nucleus block at a time,
5. FC, SD and LI matrices inside each nucleus block.

A nucleus header is parsed into

```

1 N = {
2     "index": int(m.group(1)),
3     "label": m.group(2).strip(),
4     "P": float(m.group(3)),
5     "P_formula_ok": ...
6 }

```

The final flag checks whether the text explicitly matches the expected $P = g_e g_N \beta_e \beta_N$ convention. This is important because the normalization of `nuclear_operator()` depends on that definition.

The parser also prevents a missing FC/SD/LI section of one nucleus from being silently replaced by a section belonging to the next nucleus.

5 Building the thermal level structure

5.1 make_levels

```

1 def make_levels(E):
2     levels = []
3     for s, e in sorted(E.items(), key=lambda x: x[1]):
4         if not levels or abs(e-levels[-1]["E_ref"]) >= DEGENERACY_TOL_HA:
5             levels.append({"E_ref": e,
6                             "E_values": [e],
7                             "states": [s]})
8         else:
9             levels[-1]["E_values"].append(e)
10            levels[-1]["states"].append(s)
11

```

```

12     for x in levels:
13         vals = x.pop("E_values")
14         x["E"] = sum(vals)/len(vals)
15         x["spread"] = max(vals)-min(vals)
16     return levels

```

The function separates individual state labels from physical energy levels. For each level it stores:

$$E_{\lambda} = \text{mean energy}, \quad \text{spread} = E_{\text{max}} - E_{\text{min}}.$$

The spread is printed so that the user can see whether the chosen degeneracy tolerance is reasonable.

5.2 thermal

```

1 def thermal(levels, T):
2     if T <= 0:
3         raise ValueError("Temperature must be positive.")
4     beta = 1/(KB_HA_PER_K*T)
5     E0 = levels[0]["E"]
6     b = [math.exp(-beta*(x["E"]-E0)) for x in levels]
7     Q = sum(len(x["states"])*q for x,q in zip(levels,b))
8     info = [{"x":
9             "w": q/Q,
10            "population": len(x["states"])*q/Q}
11            for x,q in zip(levels,b)]
12     return beta, info

```

Subtracting E_0 before exponentiation does not change Boltzmann ratios and improves numerical stability. w is the probability per state, while `population` is the population of the entire level.

6 Constructing the operators

The total hyperfine operator is formed by:

```

1 def add_ops(*ops):
2     return {
3         u: {
4             ij: sum(op[u].get(ij,0j) for op in ops)
5             for ij in set().union(*(op[u] for op in ops))
6         }
7         for u in "xyz"
8     }

```

so that

$$A_{\text{TOTAL}} = A_{\text{FC}} + A_{\text{SD}} + A_{\text{LI}}.$$

The perturbations are:

```

1 def field_operator(D):
2     return {u:{ij:z/C_AU for ij,z in D[u].items()}
3             for u in "xyz"}
4
5 def nuclear_operator(A, P):
6     if P == 0:
7         raise ValueError("P=0: cannot form A/P")
8     f = G_E*MU_B_AU/P
9     return {u:{ij:f*z for ij,z in A[u].items()}
10            for u in "xyz"}

```

These functions are deliberately short because they represent direct multiplicative definitions rather than independent models.

7 The central function: curie

```

1 def curie(HB, HM, info, beta, paths=False):
2     S = {(u,v): 0.0 for u in "xyz" for v in "xyz"}
3     P = []
4     for lev, x in enumerate(info):
5         for a in x["states"]:
6             for ap in x["states"]:
7                 for u in "xyz":
8                     for v in "xyz":
9                         q = x["w"] * (
10                             -beta
11                             * HB[u][(a,ap)]
12                             * HM[v][(ap,a)]
13                         ).real
14                         S[(u,v)] += q
15                         if paths:
16                             P.append((lev,a,ap,u,v,q))
17     return S, P

```

This is the physical core of the script. The loop order directly corresponds to

$$\lambda \rightarrow a \rightarrow a' \rightarrow u \rightarrow v.$$

The line

$$q = w(-\beta H_B H_\mu)$$

is one state-path contribution to one shielding-tensor element.

When `paths=True`, every contribution is stored. This makes it possible for the detailed report to show explicitly which diagonal and off-diagonal matrix elements of the populated degenerate manifold generate each component of the final tensor.

The same function is called for FC, SD, LI and TOTAL. This is useful programmatically: all components are guaranteed to use the same physical formula, and only the hyperfine operator changes.

8 Independent checks

8.1 Effective-doublet representation

For an isolated two-state ground level, the code extracts 2×2 operator blocks and expands them in the Pauli basis:

```
1 def pauli_coeffs(M):
2     a,b,c,d = M
3     return {
4         "x": (b+c)/2,
5         "y": 0.5j*(b-c),
6         "z": (a-d)/2,
7         "0": (a+d)/2
8     }
```

The corresponding doublet tensor is then compared with the direct state sum. This is only a check; the direct exact-state sum remains the primary result.

8.2 $1/T$ check

If the ground doublet is essentially fully populated at T , the program also calculates the tensor at $2T$. The check

$$\sigma(2T) \stackrel{?}{=} \frac{1}{2}\sigma(T)$$

is performed only if the ground doublet is still essentially isolated at $2T$. Otherwise it is skipped instead of reporting a false failure caused by thermal population of excited levels.

8.3 Hermiticity

```
1 def herm_error(M):
2     return max(
3         abs(z-M[u][(j,i)].conjugate())
4         for u in "xyz"
5         for (i,j),z in M[u].items()
6     )
```

This checks

$$M_{ij} = M_{ji}^*.$$

9 Long-distance Curie-only field

9.1 Dipolar geometry

For a displacement $\mathbf{R}$ from `CENTER_ATOM`,

$$\hat{\mathbf{R}} = \frac{\mathbf{R}}{R},$$

and the angular point-dipole tensor is

$$D = 3\hat{\mathbf{R}}\hat{\mathbf{R}}^T - I.$$

The code implements this through:

```
1 def dipolar_D(Rhat):
2     O = outer3(Rhat,Rhat)
3     return {
4         (a,b): 3*O[(a,b)]-(1.0 if a==b else 0.0)
5         for a in "xyz" for b in "xyz"
6     }
```

9.2 Curie B – B response

```

1 def chi0_curie(HB, info, beta):
2     S, _ = curie(HB, HB, info, beta)
3     return S

```

The same exact-state machinery is reused with both perturbations equal to H_B . The object is best interpreted as

$$Y_{BB}^{\text{Curie}},$$

not as the complete macroscopic magnetic susceptibility, because only the Curie part is present.

9.3 Long-distance scalar field

```

1 def pcs_ld_shielding_ppm(chi0, R_bohr):
2     Rn = math.sqrt(sum(x*x for x in R_bohr))
3     if Rn < 1e-9:
4         return float("nan")
5     Rhat = {u:x/Rn for u,x in zip("xyz",R_bohr)}
6     D = dipolar_D(Rhat)
7     tr = mat_trace_product(chi0,D)
8     return 1e6 * tr/(3*C_AU**2*Rn**3)

```

The implemented expression is

$$\sigma_{\text{LD}}^{\text{Curie}}(\mathbf{R}) = 10^6 \frac{\text{Tr} \left[Y_{BB}^{\text{Curie}} D(\hat{\mathbf{R}}) \right]}{3c^2 R^3}.$$

The cube is therefore a Curie-only point-dipole field. It must not be interpreted as a complete pNMR susceptibility calculation.

9.4 Cube writer

`write_pcs_cube()` validates the sign, grid spacing, padding, cutoff and centre index, generates a Cartesian grid, evaluates the field and writes a Gaussian cube.

If

`CUBE_SIGN = "shielding"`

the cube stores σ . If

`CUBE_SIGN = "shift"`

it stores $\delta = -\sigma$.

10 Output and report files

10.1 Detailed report

The file

`report/<stem>.curie_report.txt`

contains:

1. thermal levels, numerical spreads and populations,
2. total Curie shielding tensor,
3. FC/SD/LI decomposition,
4. individual state-path contributions,
5. consistency checks,
6. long-distance cube information.

This is the file to use when debugging or analysing the origin of individual tensor elements.

10.2 Compact summary

The function

```

1 def write_summary(summary_path, path, T, info,
2                   nuclei_summary, cube_summary):
3     ...

```

writes

`report/<stem>.curie_summary.txt`

with only the main results:

- full 3×3 Curie shielding tensor,
- σ_{iso} ,
- $\delta_{\text{Curie}} = -\sigma_{\text{iso}}$,
- isotropic FC/SD/LI decomposition,
- closure, doublet, $1/T$ and Hermiticity checks,
- basic cube metadata.

For routine use, this is the first file to open.

11 run() from beginning to end

The output directory is created immediately after resolving the input path:

```
1 path = Path(path).expanduser().resolve()
2 if not path.exists():
3     raise FileNotFoundError(...)
4
5 result_dir = path.parent / REPORT_DIR
6 result_dir.mkdir(parents=True, exist_ok=True)
```

The calculation then starts with

```
1 data = read_data(path)
2 levels = make_levels(data["energies"])
3 beta, info = thermal(levels,T)
4 HB = field_operator(data["D"])
```

and continues nucleus by nucleus:

```
1 for N in data["nuclei"]:
2     A = {m:N[m] for m in ("FC","SD","LI")}
3     A["TOTAL"] = add_ops(A["FC"],A["SD"],A["LI"])
4
5     for m in ("FC","SD","LI","TOTAL"):
6         S[m], paths[m] = curie(
7             HB,
8             nuclear_operator(A[m],N["P"]),
9             info,
10            beta,
11            True
12        )
13
14 R = {m:ppm(S[m]) for m in S}
```

After the per-nucleus loop:

- internal checks are performed,
- the position-independent Curie B – B response is calculated,
- the cube is written,
- the detailed report and compact summary are stored in `report/`.

The full workflow is therefore

input $\rightarrow$ parse $\rightarrow$ levels $\rightarrow$ thermal weights $\rightarrow H_B, H_\mu \rightarrow$ Curie tensor $\rightarrow$ checks $\rightarrow$ LD field $\rightarrow$ reports.

12 Current H₂O results

At

$$T = 298.15 \text{ K}$$

the ground level consists of states $[1, 2]$ and has population

$p_0 = 1.000000000$

.

Excited doublets are therefore thermally negligible in the present run.

12.1 O nucleus

The Curie shielding tensor is approximately

$$\sigma_{\text{O}}^{\text{Curie}} = \begin{pmatrix} -5458.2234 & 0 & 0 \\ 0 & -77293.6683 & 0 \\ 0 & 0 & 3727.1864 \end{pmatrix} \text{ ppm.}$$

Thus

$$\sigma_{\text{iso}}(\text{O}) = -26341.5684 \text{ ppm}$$

and

$$\delta_{\text{Curie}}(\text{O}) = +26341.5684 \text{ ppm.}$$

The isotropic decomposition is

$$FC = -22865.1612, \quad SD = +906.4625, \quad LI = -4382.8698 \text{ ppm.}$$

The result is therefore strongly FC dominated, with a substantial negative LI correction and a smaller positive SD contribution.

12.2 H nucleus

For H:

$$\sigma_{\text{H}}^{\text{Curie}} = \begin{pmatrix} 358.6972 & 0 & 2427.7709 \\ 0 & 2174.5072 & 0 \\ 2165.6475 & 0 & 1784.6907 \end{pmatrix} \text{ ppm.}$$

Therefore

$$\sigma_{\text{iso}}(\text{H}) = +1439.2984 \text{ ppm}$$

and

$$\delta_{\text{Curie}}(\text{H}) = -1439.2984 \text{ ppm.}$$

The isotropic parts are

$$FC = +1540.0885, \quad SD = -24.2953, \quad LI = -76.4948 \text{ ppm.}$$

The H shielding is again FC dominated. Unlike O, the H tensor also contains large xz/zx off-diagonal elements in the chosen molecular frame.

12.3 Numerical checks

For O:

$$|FC + SD + LI - TOTAL| = 1.46 \times 10^{-11} \text{ ppm},$$

$$|\text{direct sum} - \text{doublet}| = 9.10 \times 10^{-13} \text{ ppm.}$$

For H:

$$|FC + SD + LI - TOTAL| = 4.55 \times 10^{-13} \text{ ppm},$$

$$|\text{direct sum} - \text{doublet}| = 4.55 \times 10^{-13} \text{ ppm.}$$

The isolated-doublet $1/T$ check at 596.30 K is zero at the printed precision. These values indicate excellent internal numerical consistency.

12.4 Long-distance cube

With atom 1 (O) as the centre,

$$\text{iso} (Y_{BB}^{\text{Curie}}) = -1.498604 \times 10^{-2} \text{ a.u.}$$

The current cube has

$$25 \times 21 \times 23$$

grid points and a shielding range of approximately

$$-0.00652 \dots + 0.00419 \text{ ppm.}$$

The cube values are much smaller than the finite-nucleus O/H shieldings because the cube represents an ideal long-distance R^{-3} point-dipole field, whereas the nucleus-specific values use the actual state-resolved hyperfine matrix elements.

13 How to run and how to read the result

Default input:

```
python pnmr_curie_supervisor_report_folder.py
```

Explicit file:

```
python pnmr_curie_supervisor_report_folder.py other.data
```

Different temperature:

```
python pnmr_curie_supervisor_report_folder.py other.data --temperature 250
```

Recommended reading order:

1. Open `report/<stem>.curie_summary.txt`.
2. Read the full tensor and σ_{iso} .
3. Remember that $\delta_{\text{Curie}} = -\sigma_{\text{iso}}$.
4. Inspect FC/SD/LI isotropic decomposition.
5. Confirm that closure and Hermiticity errors are small.
6. For an isolated doublet, confirm the doublet and $1/T$ checks.
7. Use the detailed report only when state-path analysis is needed.
8. Interpret the cube only as Curie-only long-distance point-dipole output.

14 What must be changed if the input format changes

If only the molecule, number of states or temperature changes, no source-code change is required as long as the text format is the same.

If the generator changes its output:

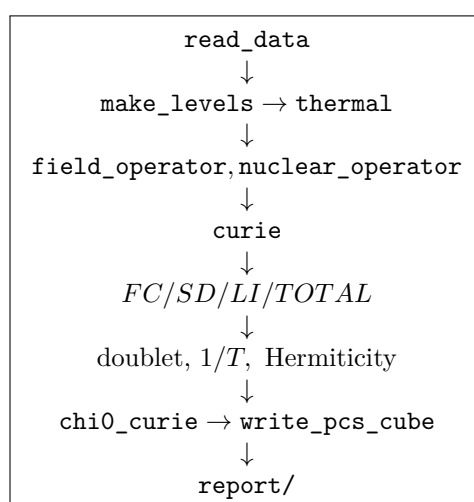
- edit `read_data()` for changed section names or nucleus headers;
- edit `matrix_block()` if the complex matrix-line format changes;
- edit `nuclear_operator()` only if the physical definition of P changes;
- edit `field_operator()` only if the supplied D definition changes;
- edit `curie()` only if the physical exact-state formula itself is changed.

Parser changes and physical-model changes should be kept separate.

15 Common interpretation mistakes

- **Shielding vs shift:** the main tensor is σ ; the corresponding Curie shift is $\delta = -\sigma$.
- **Too-large degeneracy tolerance:** may merge genuinely split levels into one Curie manifold.
- **Wrong P convention:** changes the normalization of H_{μ} .
- **LI as full orbital shielding:** incorrect; LI is only the LI hyperfine contribution.
- **Cube as complete PCS:** incorrect; this code writes only the Curie-only point-dipole field.
- **Wrong centre/geometry:** affects the cube, not the finite-nucleus exact-state Curie tensor.

16 Compact code map



17 Summary

The entire script is organised around one exact-state Curie sum. The parser builds the state-resolved operators, `make_levels()` defines the degenerate manifolds, `thermal()` supplies Boltzmann weights, and `curie()` produces the shielding tensor.

The three most important relations are:

$$\sigma_{uv}^{\text{Curie}} = \sum_{\lambda a} w_{\lambda a} \left[-\beta \sum_{a' \in \lambda} H_{B,u}^{aa'} H_{\mu,v}^{a'a} \right]$$

$$\sigma_{\text{TOTAL}} = \sigma_{\text{FC}} + \sigma_{\text{SD}} + \sigma_{\text{LI}}$$

and

$$\delta_{\text{Curie}} = -\sigma_{\text{Curie}}.$$

For routine use, the compact summary in `report/` is the main human-readable result; the detailed report is intended for diagnostics and state-path analysis.